Raspberry Pi Tech Report

Tarek Hazem/192300549/Team Logic

1. Introduction

This document describes the configuration of the Raspberry Pi 5 used in the wheelchair project. The objective is to prepare a development environment that ensures the following components are functional:

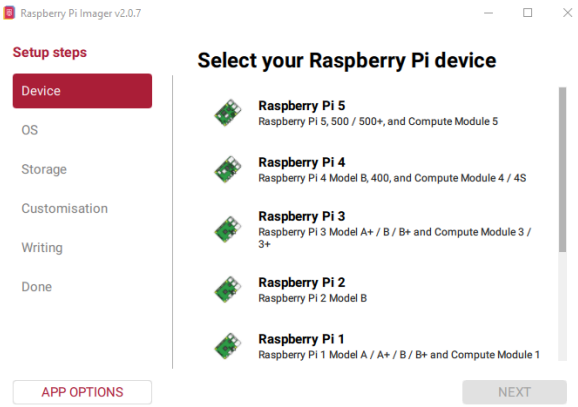
- Sensor integration (LiDAR library)
- Navigation Logic (through Python)
- UART enabled

Due to the absence of a micro-HDMI cable, this setup utilizes a headless configuration via a laptop. The following hardware is required as a bare minimum for the setup to proceed:

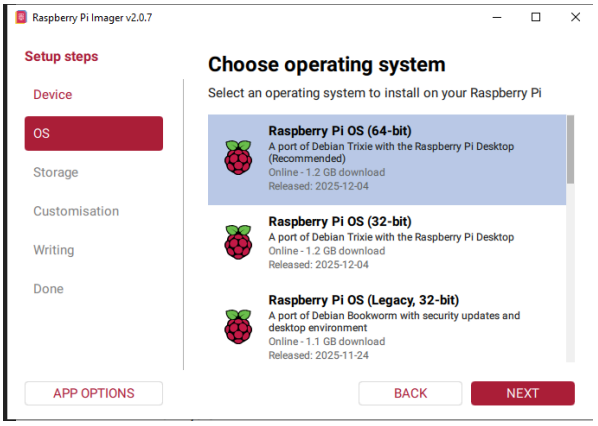
- Pi 5 (obviously)
- MicroSD Card
- Power Supply
- Laptop
- Stable Wi-Fi connection

2. Operating System Installation

The first step is to image the Raspberry Pi OS onto the microSD card, which serves as the primary storage for the device. Begin by downloading and installing the Raspberry Pi Imager from the official website. Once installed, select the Raspberry Pi 5 from the device menu:

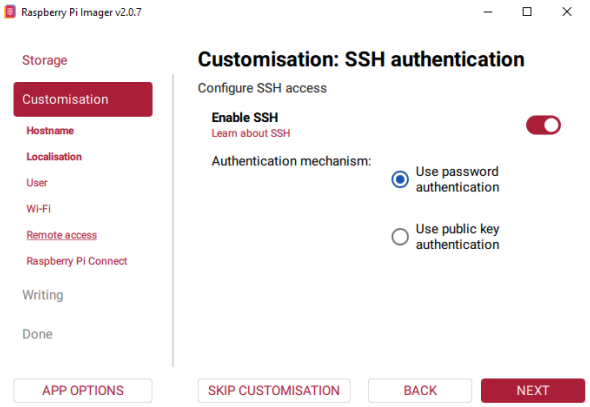


Next, select the 64-bit Raspberry Pi OS image:



After selecting the storage destination, enter the following parameters in the OS Customization options:

- Hostname: `wheelchair-pi`
- Username/Password for Pi User Account (example Username: pi/Password:raspberry)
- Network Credentials: SSID and Password for the local Wi-Fi network.
- Enable SSH (Very Important!)



-

- Optional: Enable Raspberry Pi Connect as an alternative access method.

The Imager will write the data to the microSD card. Once completed, insert the card into the Pi and power it on. A steady **green** LED indicates a successful boot.

To connect via SSH, open a terminal (or use a client like PuTTY) and execute: `ssh pi@wheelchair-pi.local`. Alternatively, locate the Pi's IP address through the router dashboard and connect using `ssh pi@[IP_ADDRESS]`.

The CMD terminal will ask for a password, and we will type the password we entered in setup.

Upon successful authentication, the terminal should display the prompt: `pi@wheelchair-pi: ~ $`. Running the `ls` command will confirm access to the Pi OS directory structure.

3. Environment Configuration

First, update the system packages using `sudo apt update`. While Python 3 is preinstalled, verify its version with `python3 --version`. Install the Python package manager by executing `sudo apt install python3-pip -y`.

It is assumed that we will need two main libraries.

- pyserial (to enable communication with the Arduino)
- rplidar

Install the required libraries using `pip3 install rplidar pyserial`. To verify the installation, enter the Python interactive shell and attempt to import the modules:

```python
import serial
print(serial.__version__)
```

```python
import rplidar import RPLidar
print("Lidar library Loaded")
```

To enable the UART interface, open the configuration tool: `sudo raspi-config`. Navigate to *Interface Options > Serial Port*. Set "Login shell accessible over serial" to **No** and "Enable serial port hardware" to **Yes**. Reboot the system to apply changes.

After all of that, we will reboot to ensure everything has installed and updated correctly.

To verify UART functionality, check for the available ports using `ls /dev/tty*`. The primary hardware serial port is typically identified as `/dev/ttyAMA0`. Test the port with the following script:

```python
import serial
ser = serial.Serial('/dev/ttyAMA0', 9600)
print("UART Functional")
```

To install ROS2, first begin by installing docker. Some distributions contained conflicting packages, so begin by typing `sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-doc podman-docker containerd runc | cut -f1)`

Then set up Docker's apt repo

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

From here, install using '`sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`' and verify using '`sudo docker run hello-world`'

After finalizing Docker, fetch and run an image of Robot Operating System 2 (ROS2) through

```
docker pull ros:humble
docker run -it --rm ros:humble
```

4. Conclusion

The Raspberry Pi 5 is now configured with the necessary operating system, remote access protocols, and Python environment. The system is prepared for sensor integration and further development of navigation logic. The ROS2 section will be used later for autonomous navigation.
SSH Test Result: Successful connection to pi@wheelchair-pi
Python Test Result: serial library imported successfully
UART Test Result: Port /dev/ttyAMA0 opened successfully